

Agents and Multiagent Systems

eBuy



MEI - 2023/2024

Francisco Claudino
PG50380



Afonso Bessa
PG53597



Rui Silva
PG54213



Group 4



University of Minho

Contents

1	Introduction	4
2	Domain Analysis and Objectives	4
3	Project Structure	4
4	Architecture	5
4.1	Initial Architecture	5
4.2	Final Architecture	5
4.2.1	Agents	6
4.2.2	Behaviours	8
4.2.3	Classes	11
4.2.4	Files Structure	12
5	Diagrams	13
5.1	Class Diagram	13
5.2	Communication Diagram	15
5.3	Activity Diagram	16
5.3.1	Purchase	16
5.3.2	Return	16
5.3.3	Supply	17
5.4	Sequence Diagram	17
5.4.1	Purchase	17
5.4.2	Return	18
5.4.3	Supply	18
6	Delivery	18
6.1	Graph	20
6.1.1	Graph Implementation	21
6.2	Algorithms	22
6.2.1	Support Structures	22

6.2.2	DFS - Depth First Search	22
6.2.3	BFS - Breadth First Search	23
6.2.4	Bidirectional Search	24
6.2.5	Uniform Cost	24
6.2.6	Iterative Search - Progressive Deepening	25
6.2.7	Heuristics	25
6.2.8	Greedy	25
6.2.9	A Star (A*)	26
6.2.10	Algorithm Comparison	26
7	Interface	27
8	Results	28
8.1	Purchase	29
8.2	Return	29
8.3	Supply	30
8.4	Negotiation	31
8.4.1	Negotiation between Client and StockManager	31
8.4.2	Negotiation between Supplier and StockManager	31
8.5	Stats	32
8.6	Addition of Client and Deliveryman	32
9	Critical Analysis and Future Improvements	32
10	Conclusion	33

List of Figures

1	Project Structure	5
2	Initial Architecture	6
3	Architecture	7
4	eBuy - Class Diagram	14
5	eBuy - Communication Diagram	15
6	eBUY - Activity Diagram - Purchase	16
7	eBUY - Activity Diagram - Return	17
8	eBUY - Activity Diagram - Supply	18
9	eBUY - Sequence Diagram - Purchase	19
10	eBUY - Sequence Diagram - Return	19
11	eBUY - Sequence Diagram - Supply	20
12	Algorithm Comparison	27
13	Interface 1	28
14	Interface 2	28
15	Failed Interface	29
16	Purchase 1	29
17	Purchase 2	29
18	Return	30
19	Supply	30
20	Client - Accept Proposal	31
21	Client - Reject Proposal	31
22	Supplier - Accept Proposal	31
23	Supplier - Reject Proposal	31
24	Stats - Moment 1	32
25	Stats - Moment 2	32
26	Add Client	32
27	Add Deliveryman	32

1 Introduction

This report serves as part of the practical project for the **Agents and Multi-Agent Systems** course within the Master's in Computer Engineering program at the University of Minho.

Multi-Agent Systems employ intelligent agents—autonomous entities capable of comprehending their environments and making decisions based on predefined objectives. These systems stem from Distributed Artificial Intelligence and involve multiple agents distributed across the system, each contributing to its overarching goals. They find application in various domains for behaviour modeling and simulation.

The practical task involved crafting a multi-agent system to handle product purchasing and returns for a company called **eBUY**, leveraging the Smart Python Agent Development Environment (SPADE) library for agent development.

This document presents an analysis of the domain, the architecture developed for this practical work, an evaluation of the obtained results and suggestions for enhancing the system.

2 Domain Analysis and Objectives

As previously mentioned, the primary goal of **eBUY** is to design and implement a multi-agent system that effectively models the processes of product purchasing and returns.

Within this domain, our system streamlines the management of purchases, returns, stock availability and the assignment of deliverymen in a structured and efficient manner. To achieve this, we introduced five essential agents: **StockManager**, **DeliverymanManager**, **Client**, **Deliveryman**, and **Supplier**.

The domain encompasses the intricate management of product transactions, spanning communication channels from the customer to the deliverymen, facilitated by the managerial roles of Stock and Deliveryman agents. Furthermore, it entails the seamless coordination of activities associated with product delivery, returns, and stock replenishment, the last overseen by the supplier.

Our system aims to establish a robust management infrastructure that streamlines product transactions and enhances customer satisfaction. This includes the ability to negotiate in cases of product unavailability and the provision of real-time information on various operations conducted by the pivotal managerial agents.

3 Project Structure

As depicted in the subsequent figure 1, the project is organized into four primary directories: Agents, Behaviours, Classes, and Comparison Algorithm. The Agents directory contains all files related to the eBuy agents. The Behaviours folder houses the specific behaviors and serialized classes used for message exchanges. The Comparison Algorithm directory encompasses the code necessary for comparing the developed algorithms, aiding in selecting the most effective one for product transportation within the system.

Additionally, there are two CSV files included: 1) to efficiently load products into the system and 2) to specify the weight of these products. Lastly, the project features the eBuy logo, a file

for the MainPage of the interface, a config.py file to store some global variables and the main.py file. This main.py file encompasses all initial setup code, adding and graphically interfacing agents and their behaviors.

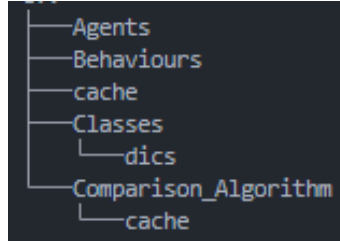


Figure 1: Project Structure

4 Architecture

In this section, a description of the developed multi-agent system will be presented, accompanied by diagrams to provide a better understanding of its architecture and operation. The architecture modeling was based on the previously described problem, attempting to conceptualize all the necessary components for the system's construction. Throughout this section, the interactions between the agents that comprise the system will also be analyzed, ensuring its integrity.

4.1 Initial Architecture

The initial concept devised by the group featured three managers, as illustrated in Figure 2. The main manager, **KingManager**, was responsible for receiving client orders and, depending on the nature of the request, forwarding them to the **StockManager** and/or **Deliveryman-Manager**. For instance, when a client inquired about available products, KingManager would contact StockManager to obtain the current product list. If the client wished to make a purchase, KingManager would then inform StockManager of the purchase request. Additionally, if negotiation was required, this would involve even more messages (this is just one example). However, as our project progressed, we realized that this approach would lead to inefficiencies. It burdened the 'messenger' agent, KingManager, with tasks beyond its primary role, resulting in unnecessary complexity. Consequently, we re-evaluated our architecture and made revisions to ensure optimal efficiency, which will be outlined in the following section.

4.2 Final Architecture

After identifying the issue with the initial proposed architecture, it was necessary to make some changes. To do so, we kept the entire structure but merged the KingManager and the StockManager into a single agent, as depicted in Figure 3.

The proposed architecture facilitates communication among the various agents, with each agent having its own function. Agents can communicate autonomously using specific messages and protocols (thanks to behaviours). This enables efficient coordination among the agents, allowing for the exchange of information necessary for the system's operation.

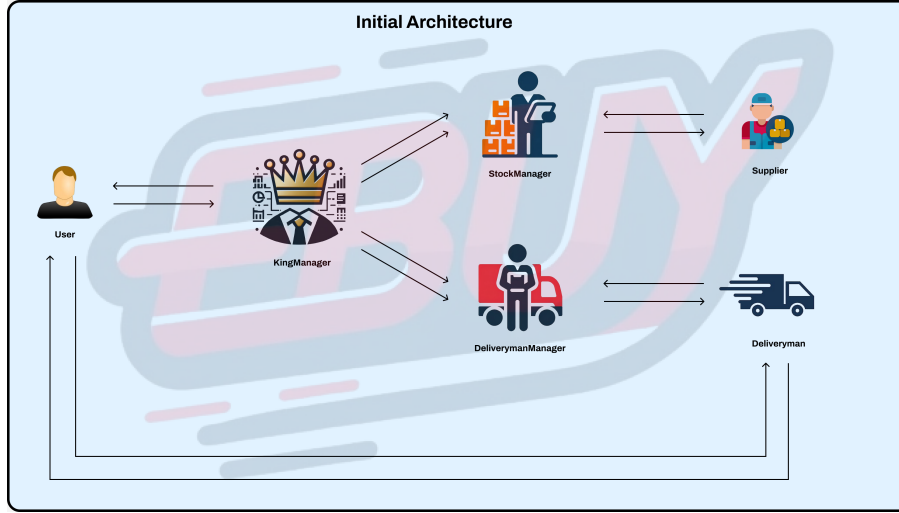


Figure 2: Initial Architecture

For the same reason, it is possible to update information in real-time: thanks to the constant communication among the agents, information can be kept updated in real-time. This has become particularly relevant for presenting statistical information and checking the availability of stock as well as deliverymen. Agents can communicate with each other immediately when relevant changes occur, ensuring the accuracy of the data.

Next, we will thoroughly explain all the agents, their respective behaviours, the created classes, as well as other files used in delivery calculations, for example.

4.2.1 Agents

As mentioned earlier, **eBUY** consists of five agents:

1. **StockManager** is responsible for overseeing and managing all warehouse stock. Additionally, it provides clients with information regarding product availability and manages both product purchases and returns. If a client attempts to purchase out-of-stock items, the StockManager will negotiate with them, and the client may accept or reject the negotiation. In case of Supplier cannot acquire all the required stock, there will be a negotiation.

As attributes the StockManager as:

- (a) Dictionaries *productsReturned* and *productsBought* to track statistics about the products bought and returned, providing a clear overview of transaction histories and product movements.
- (b) List of *products* to store product details, populated from the *stock_products.csv* file. This structured data repository enables efficient access and management of product information, facilitating operations such as restocking, sales tracking and inventory adjustments.

2. **DeliverymanManager** handles all purchase requests, selecting an available delivery person and calculating the relative order weight.

Attributes of the DeliverymanManager include:

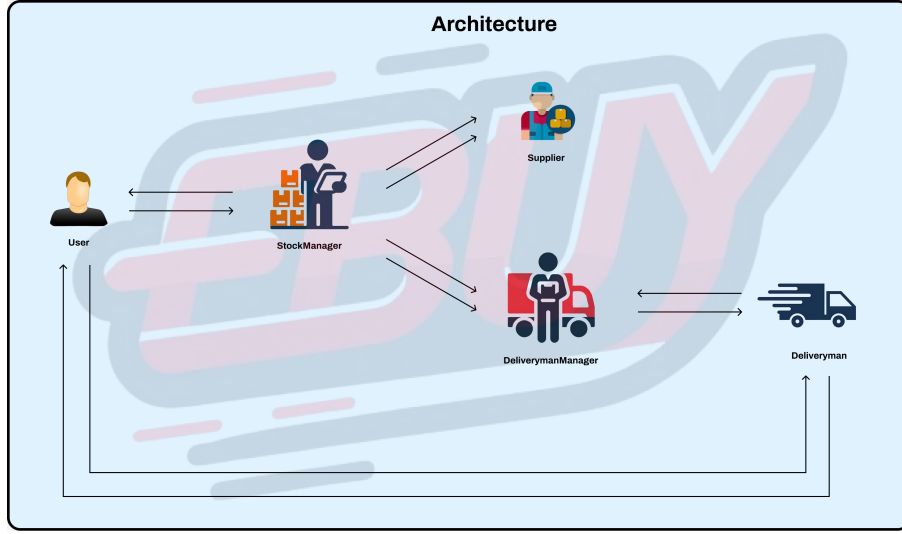


Figure 3: Architecture

- (a) *deliveryman_subscribed*: A list to keep track of subscribed deliverymen, facilitating efficient dispatch and allocation of delivery tasks.
 - (b) *products_to_be_return*: A list to track products pending return, aiding in the management of return logistics.
 - (c) *products_returned*: A list to store information on products that have been successfully returned, ensuring accurate records of returns.
 - (d) *products_to_be_delivered* and *products_delivered*: Dictionaries that respectively maintain details of products pending delivery and those already delivered.
 - (e) *products*: A dictionary to store the weight of products, indexed by product ID, read from csv file: *delivery_products*. This facilitates weight management in delivery logistics.
3. **Client** is responsible for requesting information about available products from the StockManager, purchasing products and returning purchased items. If the client attempts to purchase out-of-stock items, they will engage in negotiation with the StockManager.
- Attributes of the Client include:
- (a) *position*: Holds the geographic coordinates and node identifier, initialized randomly to simulate various Client locations.
 - (b) *productsBought*: A dictionary that records all products purchased, enabling tracking and management of purchases.
 - (c) *productsBought_notDelivered*: A list that contains products that have been bought but are still pending delivery. This helps in managing and tracking the delivery status of purchases.
4. **Deliveryman** is tasked with delivering products to customers at their location, as well as retrieving them in the case of returns. However, this depends on whether they are a **PurchaseDeliveryman** or a **ReturnDeliveryman**. Each deliveryman is assigned a specific vehicle with a maximum weight capacity. Additionally, the Deliveryman calculates the best route to deliver the products efficiently, using an appropriate algorithm that will be explained later. Before any of this occurs, they notify the DeliverymanManager of their availability, effectively announcing their availability for services.
- Attributes of the Deliveryman include:

- (a) *deliveries*: A list that tracks the deliveries assigned to the agent, ensuring that each task is managed and completed efficiently.
 - (b) *position*: Holds the geographic coordinates and node identifier, set initially to the Warehouse's location for central dispatch.
 - (c) *vehicle_type*: A variable that specifies the type of vehicle, bike, moto or car, assigned to the deliveryman, selected based on predefined probabilities to simulate different operational capabilities.
 - (d) *type*: Defines the primary action type of the deliveryman, either Purchase or Return, influencing the nature of tasks they perform.
5. **Supplier** is an agent responsible for visiting the supplier's warehouse whenever the available stock runs low. Upon being contacted by the StockManager, it goes to the warehouse and negotiates the procurement of the available stock. If unable to acquire all the required stock, Supplier will negotiate with StockManager the situation.

As attributes, the Supplier has only *position*, which provides the current location of this agent.

4.2.2 Behaviours

The agents communicate with each other through messages sent with special *performatives* and are triggered by the behaviours that each of them presents. Hence, the subsequent list outlines the behaviours of each individual agent:

Client

1. **RequestProducts_Behav** - This **PeriodicBehaviour** entails periodically sending messages to the StockManager agent to inquire about available products.
2. **ReceiveStockAndPurchase_Behav** - As a **CyclicBehaviour**, this routine manages the reception of stock information and the subsequent purchase transactions. It involves randomly selecting products for purchase based on their availability and quantity, negotiating terms with the StockManager only on special occasions, and randomly deciding to accept or reject proposals. Following the acceptance of a delivery, it updates its inventory accordingly.
3. **Return_Behav** - This **PeriodicBehaviour** facilitates the periodic return of products. It employs a method of exponential decay to determine the number of product types to return, selecting them randomly based on this calculation. The quantities for each selected product are also determined using a similar exponential decay model. After updating the his inventory, this behaviour generates and sends a return message to the StockManager.

StockManager

1. **ProcessingStock_Behav** - This **CyclicBehaviour** is essential for handling all stock-related communications with other agents. It processes incoming messages that may contain requests for product availability, purchase orders, returns and supplier proposals. Depending on the message's performative:
 - **Request**: Responds with available product details if products are requested.

- **Purchase:** Processes purchase requests, checks stock availability and either confirms the order or initiates a negotiation if stock is insufficient.
- **Return:** Manages product returns and updates stock and sales records accordingly.
- **Accept Proposal:** Handles accepted purchase proposals following negotiations, updating stock as products are sold.
- **Confirmation Refund:** Adjusts inventory for refunded products.
- **Supply:** Updates inventory based on supplier deliveries.
- **Supplier Propose:** Decides on accepting or rejecting supplier proposals based on stock needs and other criteria.

The behaviour dynamically adjusts the agent's responses based on current stock levels, incoming supply and market demands, ensuring efficient management of resources and customer satisfaction.

2. **RequestSupply_Behav** - This **PeriodicBehaviour** is implemented to ensure the StockManager maintains optimal stock levels. The behaviour periodically assesses inventory, identifying products that fall below 25% of their maximum quantity. If such products are found, it automatically generates and sends a supply request to the supplier agent. The request details the products that need replenishing, communicated via a message with the performative set to *supply*. This proactive approach helps to prevent stock shortages and ensures continuous availability of products.
3. **ProcessingSupply_Behav** - This **CyclicBehaviour** manages the reception and processing of supply updates from suppliers. Upon receiving a message with the *supply* performative, it decodes the contents to update the inventory levels. The behaviour iterates over the list of supplied products, updating the stock quantities in the StockManager's inventory to reflect the new amounts provided by the supplier. This behaviour ensures that the StockManager's inventory is always up-to-date with the latest supply changes, facilitating efficient stock management and readiness for market demands.
4. **StatsStock_Behav** - This **PeriodicBehaviour** plays a crucial role in tracking and reporting inventory statistics for the StockManager. It periodically calculates and compiles statistics for each product category based on the total quantity and price of products bought and returned. The behaviour loops through all purchased and returned products, updating statistics for each category, including the total quantities bought and returned, as well as their respective total prices. At the end of each cycle, it compiles these statistics into a comprehensive report, detailing the quantities and values of products handled, both bought and returned. This information is crucial for assessing the performance of stock management and for making informed decisions regarding inventory control and financial planning. If any significant activity has occurred, the behaviour outputs a detailed summary of these statistics, helping to maintain transparency and oversight over stock movements.

DeliverymanManager

1. **ProcessingDelivery_Behav** - This **CyclicBehaviour** is central to managing all delivery-related activities for the DeliverymanManager. It processes incoming messages with various performatives such as *purchase*, *return*, *subscribe*, and delivery confirmations.
 - **Purchase:** Upon receiving a purchase message, it calculates the total weight of the order, assigns a delivery ID and selects an available purchase deliveryman to handle the delivery. It ensures that the delivery information is packaged and sent to the designated deliveryman.

- **Return:** Processes return requests in a similar fashion by waiting for an available return deliveryman and then forwarding the return details for processing.
- **Subscribe:** Registers new deliverymen as they subscribe, adding them to the list of available personnel.
- **Confirmation Delivery:** Confirms the completion of deliveries by removing products from the pending list and adding them to the delivered list.
- **Confirmation Refund:** Manages the refund process by updating the status of returned products and communicating with the StockManager to finalize the refund.

This behaviour efficiently coordinates between the purchase and return processes, ensuring that all deliveries and returns are handled promptly and correctly by the appropriate deliverymen.

2. **StatsDeliveryman_Behav** - This **PeriodicBehaviour** that compiles and reports key statistics for delivery processes managed by the DeliverymanManager. It includes:

- Total weight of products pending delivery and delivered.
- Number of products scheduled for return and returned.
- Client with the most deliveries.
- Availability of deliverymen for purchase and return processes.

The behaviour calculates these metrics by summing weights and quantities from delivery and return lists, counting deliveries per client, and assessing deliverymen availability. Periodic reports generated by this behaviour provide essential data on delivery weights, client engagement, and staff availability, aiding in evaluating delivery efficiency and planning resource allocation.

Deliveryman

1. **RegisterDelivery_Behav** - This **OneShotBehaviour** is responsible for registering the deliveryman with the DeliverymanManager. This behaviour informs the manager of the deliveryman's operational status and readiness to receive assignments. Depending on whether the deliveryman is specialized in purchase or return tasks, a corresponding message is logged to indicate the specific focus and vehicle type of the subscription.
2. **Transport_Behav** - This **CyclicBehaviour** manages the logistics of product deliveries and returns by the deliveryman. It involves several key operations:
 - Upon receiving a *purchase* performative message, it schedules the purchase for delivery, calculating the total weight of all scheduled deliveries to determine if the delivery can be initiated based on the vehicle's capacity threshold.
 - If the total weight of the deliveries reaches at least 75% of the vehicle's maximum capacity, the deliveryman calculates the route using the configured graph based on the vehicle type (bike, moto or car), processes the delivery and sends a confirmation back to the client.
 - Similarly, for returns indicated by a *return* performative message, it processes the items to be returned, calculates the route for return and manages the logistics similar to deliveries.
 - The behaviour also involves dynamically calculating the path to the client's location for delivery or return and subsequently to the warehouse using an A* search algorithm (explained in greater detail subsequently). This includes calculating travel time, distance and managing real-time adjustments based on the trip's progress.

- After completing the deliveries or returns, the agent confirms the successful completion to the Deliveryman Manager and resets its state to available, clearing any queued deliveries or returns.

This behaviour ensures the deliveryman can handle both the delivery and return of products efficiently, managing the transport based on vehicle capacity and the optimal route calculation while maintaining communication with clients and the DeliverymanManager for updates and confirmations.

Supplier

1. **TransportSupply_Behav** - This **CyclicBehaviour** enables the Supplier to manage the logistics of transporting products between the warehouse and the supplier's warehouse. Upon receiving a *supply* message, the agent calculates the optimal route to the stock manager using an A* search algorithm over a graph representing possible paths. The steps include:

- Calculating the travel path and time from warehouse to supplier's warehouse.
- Adjusting the inventory quantities based on the stock needs — deciding randomly whether to max out the stock or propose a different quantity based on supply dynamics.
- If the proposed stock differs from the max capacity, sending a proposal back to the stock manager for approval.
- Once the supply details are finalized, calculating the return route to the warehouse, again using the A* algorithm to determine the fastest path.
- Monitoring and adapting to responses from the StockManager, including adjustments following acceptance or rejection of proposals.

The behaviour integrates logistics with dynamic inventory management, enabling the supplier to efficiently manage supply chains and respond to the stock demands of the marketplace. Detailed outputs provide feedback on the travel times and distances, ensuring transparent operation and allowing for continual adjustment of logistics strategies based on real-time conditions.

4.2.3 Classes

In this project, a variety of classes have been developed to facilitate the serialization of data for message exchanges between agents. These classes are integral for encoding and decoding state information and actions, allowing efficient communication across the system. A quick explanation of each class will be provided next.

1. **Delivery** - is a fundamental component designed to track and manage delivery details within the system. It encapsulates the delivery ID, the agent's JID (Jabber Identifier), Position class and weight of the delivery package. These features ensure that each package is monitored effectively from its origin to its destination, maintaining accurate records of its journey and characteristics.
2. **InformPosition** - is central to reporting and managing the status and position of agents within the system. It stores essential information including the agent's Jabber Identifier (JID), their geographical position (via the **Position** class), availability status and the type of agent (Purchase or Return). This setup ensures accurate tracking and dynamic management of agent assignments and locations.

3. **Position** - This class serves as a crucial component for representing spatial data within the system. It stores the coordinates **x** and **y**, along with a **node** identifier, which are essential for tracking the physical location of Clients, as well as, Supplier and Deliverymen in the environment.
4. **Product_Manager** - is specifically designed for use by Managers, Stock and Deliveryman, to manage product details comprehensively within the system. It maintains critical attributes of products, including their unique identifier, name, category, available quantity, price and maximum allowable quantity. This specialization ensures that product data handling is streamlined and precise, supporting key operational needs across managerial and logistical functions.
5. **Product** - encapsulates essential details about products in the system. It includes attributes such as the product's unique identifier, name, category and price. These attributes are crucial for the efficient management and categorization of inventory. This class is fundamental for tracking and managing products effectively, supporting operations ranging from sales to stock replenishment within the system.

Why do the **Product** and **Product_Manager** classes exist when they appear similar? The distinction is necessary because the Client should not have access to the *max_quantity* attribute, which is intended solely for the StockManager. This requirement justified the creation of two separate classes to appropriately segregate access and responsibilities.

6. **Purchase** - is designed to encapsulate all necessary details of a purchase transaction within the system. It stores the purchasing agent's Jabber Identifier (JID), the initial position of the purchase (via the **Position** class) and a list of products involved in the transaction. This class is crucial for managing purchase transactions effectively, ensuring that each transaction is accurately recorded and linked to its geographical and transactional context within the system.
7. **Return** - is specifically designed to handle the details of product returns within the system. It captures essential information about a return transaction, including the agent's Jabber Identifier (JID), the initial position (via the **Position** class) and a list of products being returned. This class plays a vital role in efficiently managing the return processes, ensuring that each transaction is accurately documented and processed according to the system's requirements.
8. **Ruas, Location, Node and Graph** - will be discussed in more detail further in the document.

4.2.4 Files Structure

To ensure streamlined data management and user accessibility, the development team opted to utilize CSV files for data storage, with JSON being another viable alternative considered. This choice was driven by the straightforward structure and ease of manipulation offered by CSV files.

The system utilizes two key CSV files:

1. **stock_products**: Managed by the StockManager, this file initializes the inventory within the *Product_Manager* class. The file structure includes columns for:

ID, Name, Category, Quantity, Price, Max_Quantity

Each column provides essential details about the products, facilitating efficient stock management.

2. **delivery_products**: Employed by the DeliverymanManager, this file is crucial for determining the weight of products, which is necessary for planning the logistics of delivery. The structure of this file is simpler:

ID, Weight

It lists each product's ID along with its corresponding weight, allowing the DeliverymanManager to accurately assess and assign delivery loads.

These structured files underpin the system's functionality, supporting the complex interactions within the multi-agent framework and ensuring that data is both accessible and manageable.

5 Diagrams

After explaining the key components of the system, four different diagrams, each with their own objective and explanation, will be presented to provide a more comprehensive understanding of **eBUY**. The diagrams will include: **Class Diagram**, **Collaboration Diagram**, **Activity Diagram**, and **Sequence Diagram**.

5.1 Class Diagram

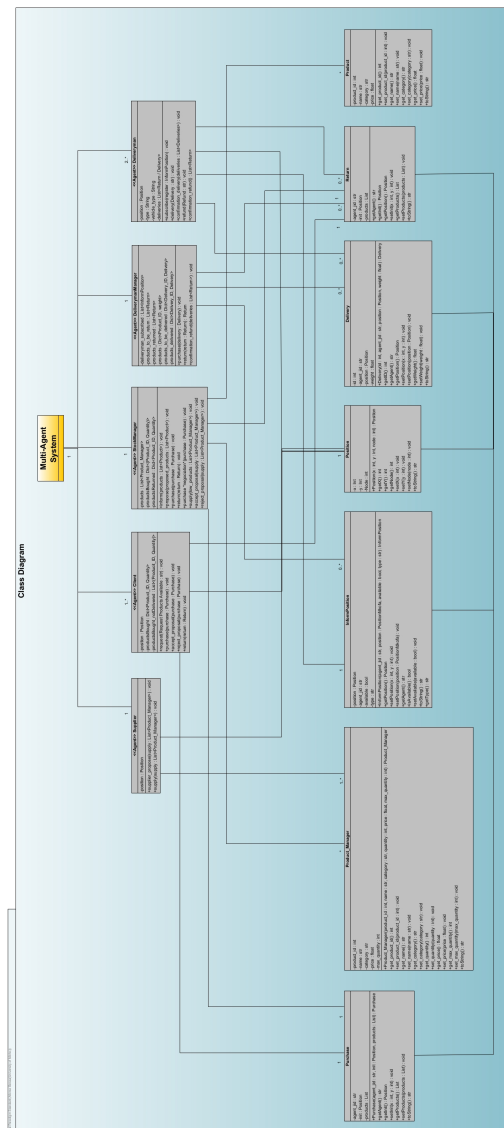
The **Class Diagram** provided below offers a comprehensive visualization of the system's architecture, illustrating the interaction and relationship between different agents and classes. This diagram is instrumental in understanding the scope of the proposed solution and the roles of various components within the multi-agent system.

As depicted in the diagram 4, the system is comprised of five distinct agents and seven classes, each serving a specific function.

It is crucial to highlight that within the system, message exchanges predominantly utilize **jsonpickle** for serialization. This approach ensures that complex data structures are transformed into a format that is appropriate for network communication. However, there are exceptions where simpler string-based messaging suffices:

- **"Request Products Available"** from Client - This direct string message is sent to the StockManager to inquire about the availability of products. Given its straightforward nature, serialization of data is not required.
- **"Delivery"** and **"Refund"** from Deliveryman - These messages communicate with the Client about the delivery of orders or the return of products. As they convey simple notifications without complex data, they do not utilize serialized classes. These messages are designed for clear and efficient communication about the status of orders.

This strategy ensures that while **jsonpickle** is used for most communications to handle complex data exchanges efficiently, simpler messaging formats are employed where feasible to optimize communication efficiency.



In the system, there can be one or more Clients as well as two or more Deliverymen. However, the remaining agents — StockManager, DeliverymanManager and Supplier — can only exist as one and only one of each.

5.2 Communication Diagram

Subsequently, a **Communication Diagram** (or Collaboration Diagram) was developed, which has provided a clear understanding of how the agents interact and the key behaviors that have been implemented. This diagram effectively illustrates the communication paths between the agents, detailing the interactions that drive the **eBUY**'s functionality.

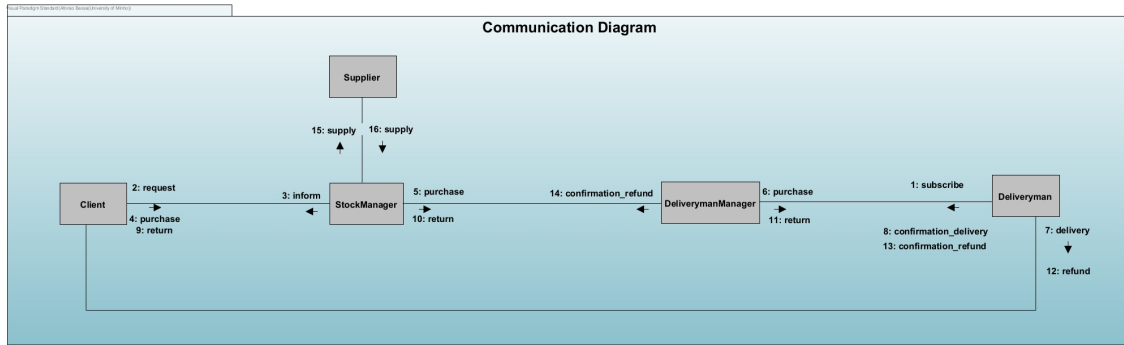


Figure 5: eBuy - Communication Diagram

We can categorize the operations in the **Communication Diagram** into three distinct phases: **Purchase**, **Return** and **Supply**, as depicted in Figure 5.

Purchase

The initial interaction in the Purchase phase starts when the Deliveryman *subscribes* to the DeliverymanManager, indicating their readiness to accept delivery tasks. Following this setup, the Client sends a *request* to the StockManager inquiring about the products available at **eBUY**. The StockManager *informs* the Client of the available stock. After considering the options, the Client proceeds to make a purchase, placing an order and communicating this decision back to the StockManager. The StockManager then forwards these order details to the DeliverymanManager, who assesses the order's weight and assigns a Deliveryman from the subscribed list. The chosen Deliveryman *delivers* the products to the Client's location. Once all deliveries are completed, the Deliveryman sends a *confirmation* message back to the DeliverymanManager to signify the completion of all tasks.

Return

Should a Client decide to *return* a product, possibly due to damage or dissatisfaction, this initiates the Return phase. The Client informs the StockManager of the products to be returned. The StockManager then communicates this *return* request to the DeliverymanManager, who assigns a Deliveryman to handle the return in a manner similar to the purchase process. The Deliveryman collects the items from the Client and, after collecting all returns, sends a *confirmation* message to the DeliverymanManager. This manager, in turn, notifies the StockManager to confirm the receipt of the returned items, allowing for the appropriate adjustments in the inventory if the products are in a suitable condition.

Supply

The Supply phase occurs when inventory levels are low, necessitating a restock from a Supplier. The StockManager initiates contact with the Supplier to request additional supplies. The first communication in this phase, labeled *supply*, informs the Supplier about the needed products. The Supplier then delivers these items and the second communication ensures that the StockManager receives updates about the replenished stock levels. This phase ensures the system maintains an adequate supply to meet future client demands.

5.3 Activity Diagram

An Activity Diagram provides us with a clearer understanding of the main operational flows and interaction requirements among the different system agents. As demonstrated in the previously presented diagram, it can be subdivided into three phases: **Purchase**, **Delivery** and **Supplier**.

5.3.1 Purchase

As detailed previously, this diagram 6 outlines the purchase process from the initial product inquiry by the Client to the final delivery by the Deliveryman. It includes the negotiation phase, which occurs if there is a stock shortfall, ensuring that any issues are addressed before the order completion.

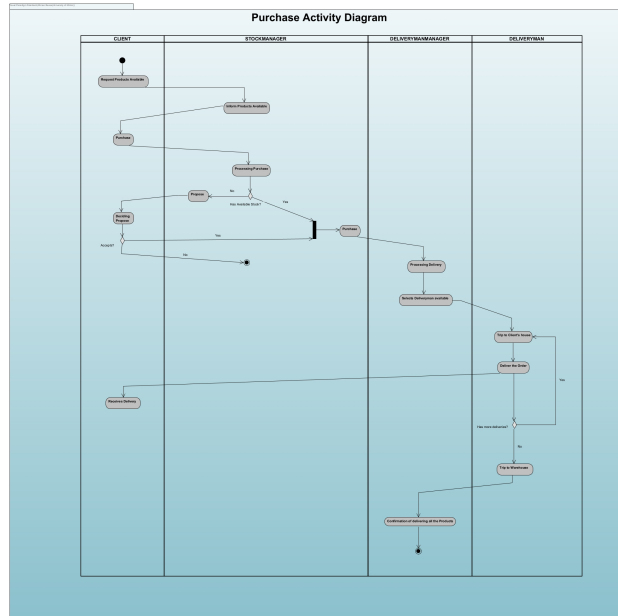


Figure 6: eBUY - Activity Diagram - Purchase

5.3.2 Return

Similarly, the subsequent diagram 7 illustrates the process of returning purchased products, from the initiation by the Client to the collection by the assigned Deliveryman, categorized as a *ReturnDeliveryman*. Following the collection, the StockManager updates the inventory based on the condition of the returned items. This update is guided by predefined probabilities where, for

example, items like Clothing have a significantly higher chance of being restocked compared to consumables like Drinks or Food.

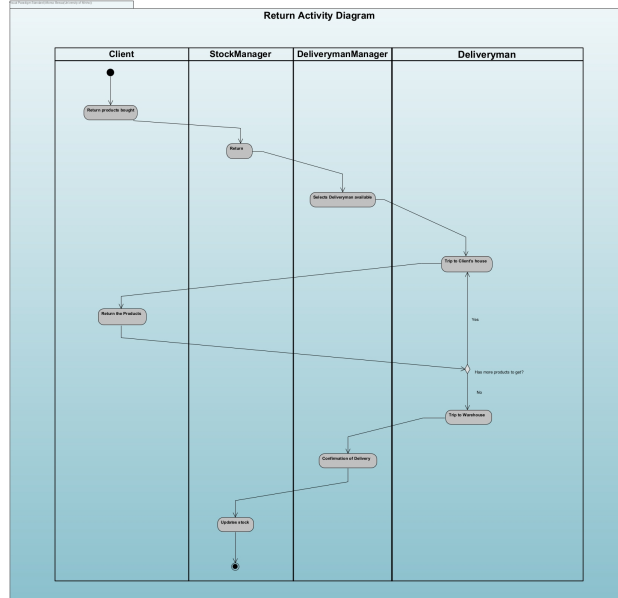


Figure 7: eBUY - Activity Diagram - Return

5.3.3 Supply

Lastly, the diagram in reference 8 details the complete supply process, including scenarios of stock availability. If the supplier's warehouse has the full quantity of requested products available, these items are delivered directly to the StockManager at the Warehouse without the need for further negotiation. In cases where not all requested products are available, the process involves a negotiation phase. If the StockManager accepts a partial supply, the Supplier proceeds to deliver the available products, which allows the StockManager to update the inventory accordingly. If no agreement is reached, the Supplier returns to the warehouse without delivering any products.

5.4 Sequence Diagram

The Sequence Diagram plays a crucial role in illustrating the dynamic behavior of the system through interactions between different entities over time. Each diagram captures the sequence of messages exchanged among the agents, highlighting the key performatives used in the communication processes of purchasing, returning and supplying operations.

5.4.1 Purchase

This **Sequence Diagram 9** specifically focuses on the purchase process. The primary performatives used are *purchase* and *delivery*. The *purchase* performative is passed from the Client to the StockManager and subsequently to the DeliverymanManager. The *delivery* performative pertains to the transportation of products, initially from the DeliverymanManager to the selected Deliveryman and subsequently from the Deliveryman to the Client. This diagram also illustrates the

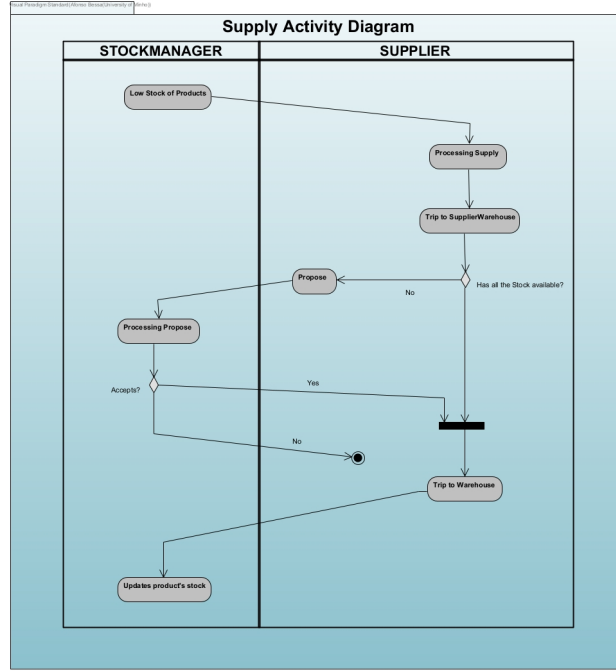


Figure 8: eBUY - Activity Diagram - Supply

negotiation process from Client and StockManager with performatives being *propose* and *accept* or *reject proposal*.

5.4.2 Return

The Return Sequence Diagram 10 illustrates the entire product return process. The most frequent performative in this process is *return*, with the object passed between agents being either a *Return* instance or a *list of Returns* in the case of return confirmation.

5.4.3 Supply

Finally, this diagram 11 illustrates the supply process, which is initiated when the StockManager detects stock levels falling below 25% of maximum capacity. The messages exchanged between Supplier and StockManager always involve objects of type *Product_Manager*, whether for negotiation or the actual delivery of products.

6 Delivery

In the initial phase of the project, during the development of the core components (Agents, behaviours, and Classes) and the architecture of **eBuy**, we opted for a 100 by 100 grid framework. This approach was modeled after the method used in the practical classroom exercises, specifically the Taxi worksheet. During this phase, we utilized Euclidean distance to calculate delivery times, estimating the time it would take to reach a customer's coordinates (x, y).

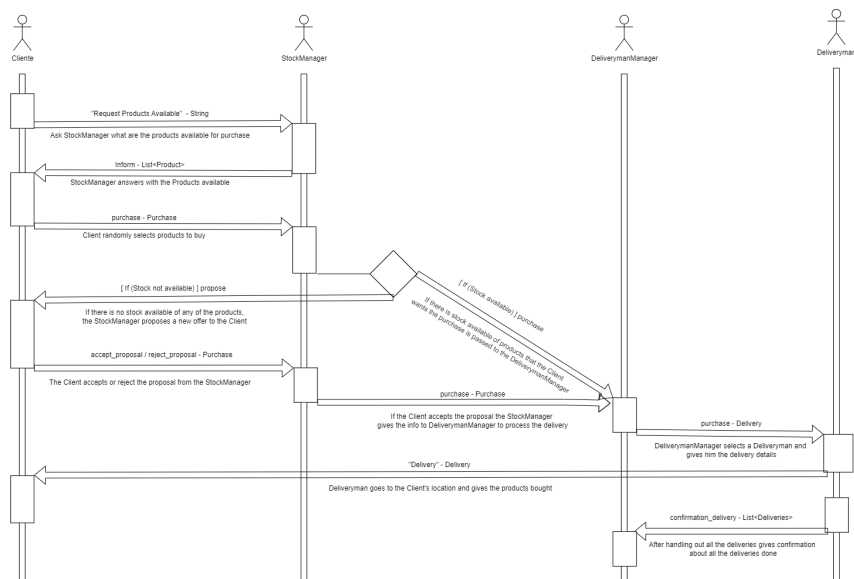


Figure 9: eBUY - Sequence Diagram - Purchase

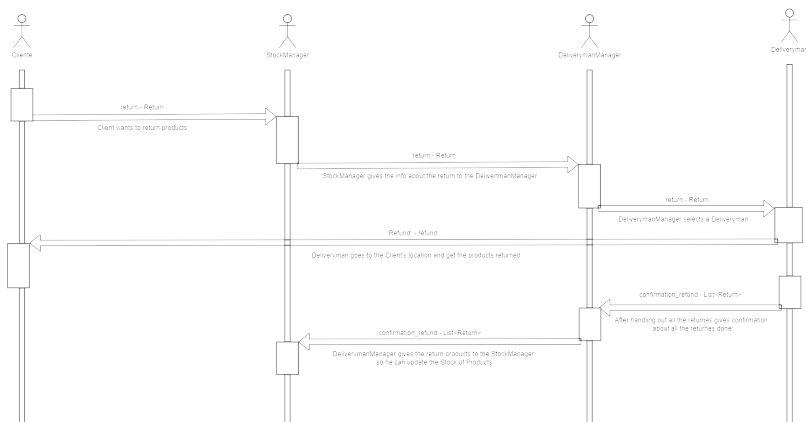


Figure 10: eBUY - Sequence Diagram - Return

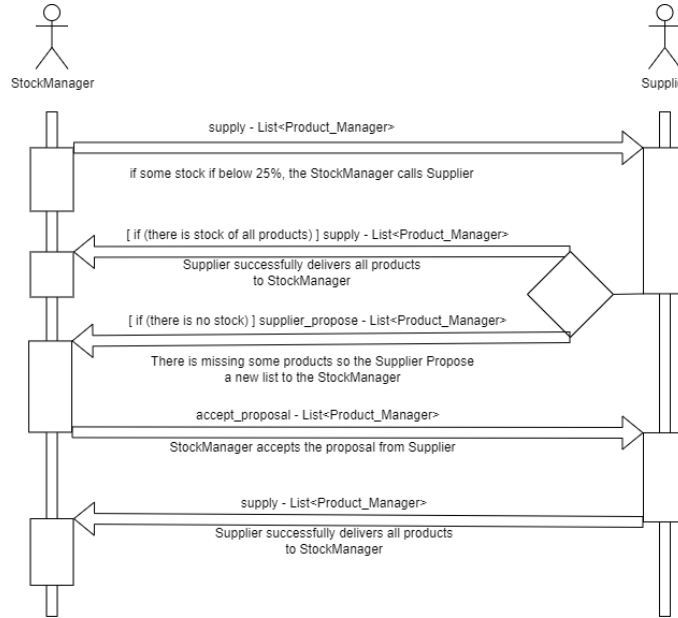


Figure 11: eBUY - Sequence Diagram - Supply

However, this method is not viable for a realistic, dynamic or scalable application. The simplified grid model and the use of Euclidean distance do not accurately reflect real-world delivery times, as they fail to account for real-world travel conditions and obstacles.

6.1 Graph

After some deliberation and research, a solution was found that aligned with the group's ambition to implement a realistic and feasible resolution. This solution took shape with the discovery of the **OSMnx** [2] library, a Python package for downloading, modeling, analyzing, and visualizing street networks and other geospatial data from **OpenStreetMap**. Created by Geoff Boeing, a professor at the University of Southern California, the *nx* in OSMnx refers to its construction on top of the **NetworkX** [3] library, which is useful for studying the structure and dynamics of complex networks/graphs.

Using this package allows us to expand our search universe to a global level, making the application usable in potentially every location worldwide. All of this is achievable with just a line of code.

```
def run(location):
    # Download the street network
    G = ox.graph_from_place(location, network_type="drive")
```

The *graph_from_place* method accepts a parameter specifying the type of network, which can be one of the following options:

1. **drive** - retrieves public roads suitable for driving, excluding service roads.
2. **drive_service** - retrieves all drivable public roads, including service roads.

3. **walk** - retrieves all roads and paths accessible to pedestrians, ignoring one-way street restrictions.
4. **bike** - retrieves all roads and paths usable by cyclists.
5. **all** - retrieves all roads and paths from OpenStreetMap.

The graph returned by the OSMnx library is a weighted MultiDiGraph, which is a directed multigraph, meaning there can be multiple edges between the same nodes. In this graph, edges represent roads or segments connecting nodes; the direction of the edges indicates the flow of traffic, and they also contain information about their length, road type, section number, among other details. This library is rich in content that could be utilized in a wide range of projects, but for our purposes, we will only explore a few specific aspects that will be discussed later.

6.1.1 Graph Implementation

Location.py

It was decided that some filtering of the information provided by the OSMnx library was necessary, as it was not relevant to our specific problem. Therefore, we removed data related to the nodes, specifically their names, sizes, whether they are part of a highway or not and their reference codes. For the edges, we removed information regarding their OSM identifier, whether the direction is reversed, their geometric shape, and the number of lanes going in the same direction, particularly if it is a junction.

There was also a need to edit the names of some streets since a street could appear multiple times in a potential list of edges. For example, an avenue may have several intersections along its route; in this graph, a different edge is represented each time a street is *interrupted* by an intersection.

The solution the group formulated was to insert an indicator in the name of each edge that reflected the order of the street section. To conclude this part of the graph formulation, it remains to mention the creation of structures that will store the nodes, edges and their sets, which will be utilized by the Graph class to be explored later.

The three data structures that will be part of the Graph definition are:

- **Graph Dictionary** - A dictionary representing each node and its adjacent nodes.
- **Edges List** - A list of all the edges between nodes.
- **Nodes List** - A list of all the nodes.

In the method *create_neighborhood_dict*, we generate a dictionary where the keys are the identifiers of a node, and the value is a list containing relevant information about the node's adjacent nodes.

In the *create_node_list* method, we produce a simple list of node objects, which includes their latitudinal and longitudinal coordinates and the number of streets that intersect them.

Particular attention was needed for handling edges, given the variety of street types, such as one-way streets, highways, bridges and tunnels. Additionally, one of the dynamic aspects of the project is implemented through the *randomizacao_de_cortadas_transito* method, which assigns a

10% probability to each scenario of streets being either excessively trafficked or completely closed. Notably, a street cannot be simultaneously marked as having heavy traffic and being closed.

6.2 Algorithms

After assembling the graph with all the streets for a given location, a crucial phase remains. What is the best algorithm for optimizing the deliveryman's route during product delivery or return by the Client, as well as for the supply of products by the Supplier? To address this, seven different algorithms were applied to determine which is most ideal and appropriate for the specific problem.

Next, we will explain each of these algorithms and, ultimately, present the performance of each to identify the most effective algorithm chosen.

6.2.1 Support Structures

To facilitate the management of the large number of nodes and edges and their respective information returned by the **OSMnx** package, it was necessary to create some classes and modify others.

The **Node** class represents intersections between streets. It has an identifier provided by the **OSMnx** package, an instance of the **Positions** class that represents the latitudinal and longitudinal positions of the respective node and a counter of how many streets intersect at the node.

The representation of streets themselves is done by the **Ruas** class, referring to the edges that connect the nodes in our multigraph.

Attribute	Type	Description
name	string	Name of the street
node_origin	int	Identifier of the origin node
node_destination	int	Identifier of the destination node
oneway	bool	Whether the street is one-way or not
highway	string	Type of road
roundabout	bool	Whether it is a roundabout or not
bridge	bool	Whether it is a bridge or not
tunnel	bool	Whether it is a tunnel or not
max_speed	float	Maximum speed on a given street
length	float	Length of the street
cut	bool	Whether the road is closed or not
traffic	bool	Whether the road has traffic or not

Table 1: Attributes of the Edges

6.2.2 DFS - Depth First Search

Depth-First Search (DFS) is a systematic graph exploration approach that prioritizes exploring as deeply as possible before backtracking. Given the potential for our graph to be extensively

large, this algorithm may exhibit low performance, specifically due to its slow solution-finding time; further details on these performance metrics will be provided subsequently.

The function begins by initializing a list called `path` to store the current route being explored. The starting node is added to this list and designated as visited. The function subsequently checks if the starting node coincides with the destination node. If they are identical, the discovered path is returned along with the total path cost and the aggregate number of nodes explored. If the starting node differs from the destination node, the function continues to examine the nodes adjacent to the starting node. For each adjacent node, it verifies whether the node has been visited previously and whether the corresponding street is open. If these criteria are satisfied, the function recursively calls itself to explore the adjacent node.

During this exploration, the function takes into account streets experiencing traffic, incorporating them into a list if they are encountered along the route. This monitoring of traffic-laden edges facilitates the adjustment of passage costs if the route traverses a traffic-affected street.

If the recursive invocation uncovers a path, the outcome is promptly returned, halting further exploration of additional branches. If not, the current node is unmarked as visited, removed from the path and the function yields `None`.

This depth-first exploration process persists until all conceivable paths have been evaluated or a valid path is identified. The function returns the path, its total cost and the complete count of nodes explored, or `None` if no path emerges. This implementation encapsulates the recursive and methodical essence of the depth-first search algorithm.

6.2.3 BFS - Breadth First Search

Breadth-First Search (BFS) is an algorithm that relies on progressively exploring neighboring nodes before moving on to more distant ones.

The function starts by marking the starting node as visited and placing it in a queue for processing. Next, a loop is initiated to explore the nodes in breadth. During this process, the function examines the nodes adjacent to the current node, checking if they have been visited and if the corresponding street is not blocked. If the adjacent node meets these conditions, it is added to the queue for future exploration.

The algorithm continues this process until the queue is empty or a valid path to the destination node is found. The function maintains a record of the parents of each visited node, allowing for the reconstruction of the path later on, as well as keeping a list of streets with traffic that can be used to calculate the correct cost of passing through heavily trafficked streets.

The total cost of the path is calculated considering the costs associated with traffic edges. Ultimately, the function returns a tuple containing the reconstructed path, the total cost and the total number of nodes explored.

This implementation of breadth-first search is efficient and organized, offering a systematic approach to finding paths in a graph, taking into account factors such as traffic edges and associated costs.

6.2.4 Bidirectional Search

Bidirectional Search is an approach that simultaneously explores from both the starting node and the ending node, meeting at a midpoint.

The function initiates two queues, one for the search from the starting node to the meeting point (forward search) and another for the search from the destination node to the meeting point (backward search). It also maintains sets of visited nodes (*forward_visited* and *backward_visited*) and dictionaries of parents (*forward_parent* and *backward_parent*) for path reconstruction.

The algorithm continues execution as long as both queues are not empty and a valid path has not yet been found. During each iteration, it explores the nodes adjacent to the current ones in both directions. If there is an intersection in the sets of visited nodes, a meeting point has been found, indicating that the bidirectional search was successful. The function logs the meeting point and activates the *path_found* flag.

During exploration, the function checks if the adjacent nodes have not been visited and if the corresponding edges are not blocked. Valid adjacent nodes are added to the queues for future exploration, their parents are recorded, and the sets of visited are updated.

After finding the meeting point, the function reconstructs the path from the parents in both directions using the *forward_parent* and *backward_parent* dictionaries.

Finally, the function calculates the total cost of the path considering the traffic edges encountered during the search and returns a tuple containing the reconstructed path, the total cost, and the total number of nodes explored.

To enable exploration from the end to the beginning, it is also necessary to explore the neighbors in reverse, that is, to explore the nodes where one of the adjacents is the current node. For this purpose, the method *get_predecessors* was implemented.

Similarly, special reconstruction of the obtained path is necessary since the order of the nodes within the dictionaries is reversed. To address this, the method *reconstruct_path_bidirectional* was implemented.

6.2.5 Uniform Cost

Uniform-Cost search is an algorithm that prioritizes exploring nodes with the lowest accumulated cost so far. The implementation utilizes a priority queue (*min-heap*) to keep track of movements with lower costs.

The function initiates a priority queue with a tuple containing the accumulated cost up to the current node and the starting node itself. A set of visited nodes is maintained, along with a dictionary of parents (*parents*) for path reconstruction. The algorithm continues to run as long as the priority queue is not empty and a valid path has not yet been found. During each iteration, the node with the lowest accumulated cost is removed from the priority queue. If the current node is the destination node, the search has been successful, and the function activates the *path_found* flag.

During exploration, the function checks whether the adjacent nodes have not been visited and whether the corresponding edges are not blocked. Valid adjacent nodes are added to the priority queue, their parents are recorded, and the set of visited is updated. After finding the destination node, the function reconstructs the path using the parents from the parents dictionary.

Finally, the function calculates the total cost of the path, taking into account the traffic edges encountered during the search, and returns a tuple containing the reconstructed path, the total cost, and the total number of nodes explored.

6.2.6 Iterative Search - Progressive Deepening

Iterative Search - Progressive Deepening is a search strategy that performs depth-first searches with incrementally increasing depth limits. The function *procura_iterativa* initiates a loop that progressively increases the depth limit until a path is found.

Within the loop, the function calls *procura_iterativa_ciclo*, which is essentially a copy of the depth-first search (DFS) algorithm with some additional checks. This function receives the current node, the destination node, the current depth limit, the current path, a set of visited nodes, a list of traffic edges, and the total number of nodes explored so far. If the current node is the destination node, the function returns the found path, the list of traffic edges, and the total number of nodes explored. If the depth limit reaches zero, the function returns *None*, indicating that the depth limit was reached without finding a path.

During the exploration, the function checks whether the current node has not been visited and whether the corresponding edge is not blocked. Valid nodes are added to the path, to the list of visited, and to the list of traffic edges. Subsequently, the function makes a recursive call with the adjacent node. After finding a path, *procura_iterativa* calculates the total cost of the path considering the traffic edges encountered during the search and returns a tuple containing the reconstructed path, the total cost, and the total number of nodes explored.

6.2.7 Heuristics

Initially, the group considered using a heuristic based solely on direct distance (Manhattan distance). However, this approach presents certain challenges. For instance, two intersections may be geographically close yet require a significantly longer actual travel route due to road configurations, leading to a significant difference between the direct line-of-sight distance and the actual travel distance on the ground.

Consequently, the group opted for a more detailed method that more accurately reflects real-world conditions. A technique similar to breadth-first search was employed. Starting from the destination node, adjacent nodes are explored and added to the heuristic dictionary with the nodes as keys and the travel time from the destination to these nodes as values. Therefore, it became necessary to maintain a separate dictionary for each type of vehicle, as each has a different average speed (*heuristics_by_vehicle*).

6.2.8 Greedy

The *greedy* function implements the **Greedy** algorithm, an informed search that selects the node with the lowest heuristic at each step, without considering the accumulated cost to that point. The function begins with an *open_list* containing the starting node and an empty *closed_list*. Additionally, it maintains a set of transit edges (*transito_list*), a *parents* dictionary that records the parents of each node, and a variable *n_nos_explorados* that counts the total number of nodes explored.

The main loop continues as long as the *open_list* is not empty. At each iteration, the node

with the lowest heuristic is selected from the *open_list* for exploration. If the current node is the destination node, the algorithm reconstructs the path from the parents and returns a tuple containing the reconstructed path, the total cost, and the total number of nodes explored. For each neighbor of the current node, the algorithm checks if the neighbor is not in the *open_list* or the *closed_list*. If not, the neighbor is added to the *open_list*, and its parents and transit edges are updated.

If the neighbor is already in the *open_list* or *closed_list*, the algorithm checks if it is more efficient to reach the neighbor through the current node. If so, the parents and transit edges are updated. After exploring all neighbors of the current node, it is removed from the *open_list* and added to the *closed_list*. The algorithm continues until the *open_list* is empty or the destination node is found.

6.2.9 A Star (A*)

The *procura_aStar* function implements the **A*** algorithm, an informed search that uses a combination of current cost (g) and heuristic (h) to determine the order of node exploration. The function starts with an *open_list* containing the start node and an empty *closed_list*. Additionally, it maintains a set of transit edges (*transito_list*), a g dictionary that stores the accumulated costs up to each node, a parents dictionary that records the *parents* of each node, and a variable that counts the total number of nodes explored.

The main loop continues as long as the *open_list* is not empty. Each iteration removes the node with the lowest evaluation function value $f()$ (the sum of the current cost $g()$ and the heuristic $h()$) from the *open_list* for exploration. If the current node is the destination node, the algorithm reconstructs the path from the parents and returns a tuple containing the reconstructed path, the total cost, and the total number of nodes explored.

For each neighbor of the current node, the algorithm checks whether it is not already in the *open_list* or *closed_list*. If not, the neighbor is added to the *open_list*, and its parents, accumulated costs, and transit edges are updated. If the neighbor is already in the *open_list* or *closed_list*, the algorithm assesses whether reaching the neighbor through the current node is more efficient. If so, the parents, accumulated costs, and transit edges are updated. After exploring all the neighbors of the current node, it is removed from the *open_list* and added to the *closed_list*. The algorithm continues until the *open_list* is empty or the destination node is found.

6.2.10 Algorithm Comparison

The variable *n_nos_explorados*, returned by each algorithm, is used along with other statistics to measure the performance of each. The chosen parameters for this measurement are:

1. **Execution Time**
2. **Number of Function Calls Made**
3. **Length of the Path Found**
4. **Distance**
5. **Time**
6. **Number of Nodes Explored**

For this purpose, the **cProfile** library was utilized to measure execution time and function calls. Additionally, the **pstats** library was necessary to extract essential information from the profilers.

To illustrate, we will use an example of a search between **Rua de Souselas** and **Rua Bernardo Sequeira** in the city of **Braga**.

Algoritmo	Tempo de Execução (s)	Número de Chamadas de Funções	Tamanho do Path	Distância (m)	Tempo (s)	Número de Nós Explorados
DFS	139.47	389438722	335	26723.73	1924.11	334
BFS	21.66	47454359	34	3451.33	248.5	1510
Bidirecional	14.79	28954277	34	3451.33	248.5	699
Custo Uniforme	11.38	25196416	39	3854.92	219.95	983
Procura Iterativa	275.93	567491797	55	6487.6	467.11	54
Greedy	3.19	5593677	55	4769.02	343.37	153
A*	8.16	16276443	38	2886.86	207.85	408

Figure 12: Algorithm Comparison

From these results, we can conclude that while the Greedy algorithm is the fastest, it is A* that consistently delivers the best solution.

As the Greedy algorithm does not consider cost, it is notable that despite its rapid execution, the distance obtained is considerably higher than that achieved by A*. In our case, this is because the Greedy algorithm does not take into account the actual lengths of the streets.

Thus, all transport operations for product delivery, return, and supply will utilize the A* algorithm.

7 Interface

Once the entire system was functional, we reached the stage of implementing a graphical user interface for **eBUY**.

After some thought and discussion about what to implement, we decided to start with a *MainPage* where the user would have the option to choose any location, number of clients, and number of deliverymen.

Thus, using the **Tkinter** library, the first interface was developed, as shown in Figure 13.

To provide users with a better experience, it was necessary to implement an interface capable of displaying the behavior of each agent. Initially, using the **Streamlit** library, we attempted to implement a separate window for each agent so that each could display its unique behaviors. However, after unsuccessful attempts, it was necessary to find an alternative. Returning to **Tkinter**, we implemented Interface 2, as shown in Figure 14.

Even after this interface was fully functional, we aimed to further enhance it by implementing a *notebook* style (Figure 15), as it is referred to in **Tkinter**, allowing each Agent to have its respective interface. However, after numerous errors, this attempt was eventually abandoned by

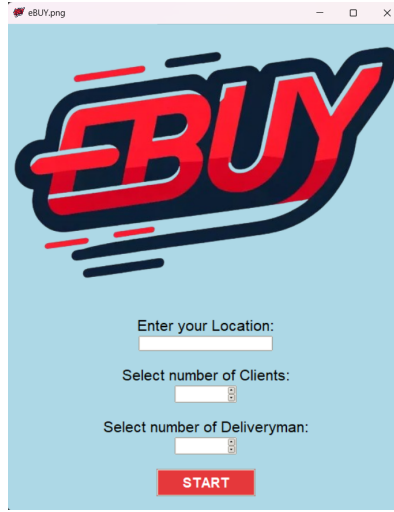


Figure 13: Interface 1

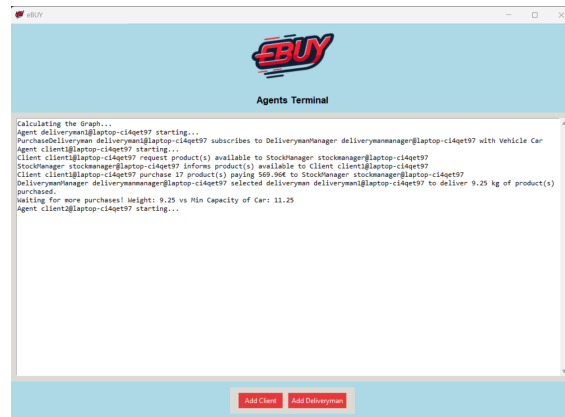


Figure 14: Interface 2

the team.

8 Results

This section presents the results of a test simulation conducted on various agents and behaviors within the **eBUY** system.

As previously mentioned, the system allows the use of any number of Clients and Deliverymen. For the purposes of this demonstration, we will focus on the county of Sabrosa in Vila Real, utilizing 3 Clients and 3 Deliverymen to more effectively illustrate the results obtained.

Another point to note is that in order to display the results, we will adopt a `time.sleep(2)` for the delivery of each order, although it is possible to check the time needed to complete the journey, as we will see shortly.

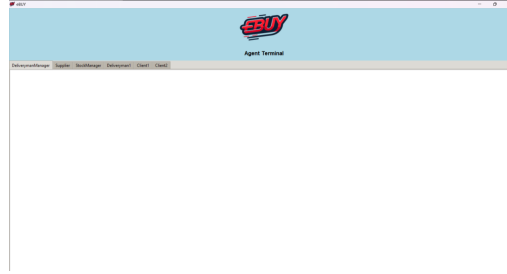


Figure 15: Failed Interface

8.1 Purchase

As we can see in Figures 16 and 17, initially, the respective graph is calculated, agents are initialized and the deliverymen subscribe to the DeliverymanManager's list of available deliverymen. Following this, Client1 makes a "Request Products Available" to the StockManager, who responds accordingly. Subsequently, this same Client makes a purchase of three products totaling 227.11 euros. After informing the DeliverymanManager of the order, which weighs 1.93 kg, Deliveryman1 is selected. Since the vehicle of the Deliveryman, in this case, a Bike, can still carry more weight, it waits for another order. Once Client2 is initialized, the same purchase process occurs. As the weight exceeds the bike's minimum load capacity, the delivery is executed.

Thus, the first path from the Warehouse to Client1 is calculated using the A* algorithm as previously mentioned. This route would take about 43 minutes and cover 11 kilometers by bike. Similarly, the entire path from Rua Central de Roalde to Estrada Municipal 323 is detailed. After the delivery is completed, the process is repeated, but this time from the location of Client1 to Client2, located on Rua da Madelena. After all deliveries are completed, it is necessary to return to the Warehouse, a journey of about 6 minutes covering just over 1 kilometer. Upon arrival, the Deliveryman informs the DeliverymanManager that all deliveries have been completed and that he is again available.

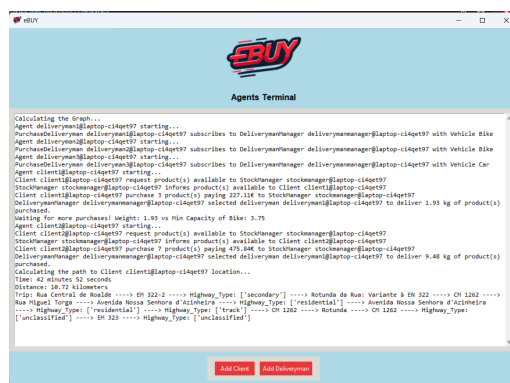


Figure 16: Purchase 1

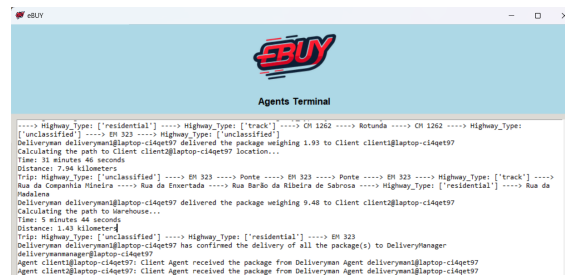


Figure 17: Purchase 2

8.2 Return

As depicted in Figure 18, there is a mechanism in place that waits for the accumulation of products before proceeding with returns; in this instance, the system waits for five products to be ready for return. After a client requests a return exceeding this threshold, a deliveryman is assigned

to handle the return of these products. Similar to the purchase process, the delivery route to the client's location for returns is calculated. Once the returns are completed, the deliveryman heads back to the warehouse, confirms the completion of all product returns to the DeliverymanManager, who in turn notifies the StockManager. The StockManager then updates the stock levels for the returned products.

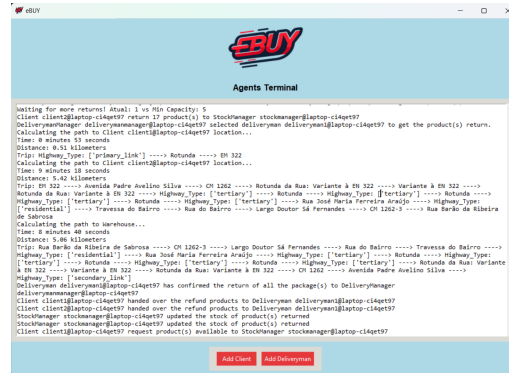


Figure 18: Return

8.3 Supply

In the case of Supply, a simulation is conducted where, upon initializing the order, there is no stock for some products to easily trigger this behavior.

As illustrated in Figure 19, five products were missing, prompting the StockManager to request a *supply* from the Supplier. The Supplier then travels from the Warehouse to the SupplierWarehouse, navigating through various streets and squares such as Largo Rocha Cabral and Rua da Escola. After loading the full stock, the Supplier returns to the Warehouse. This return journey takes approximately 17 minutes, covering 14 kilometers. Upon arrival, the Supplier informs the StockManager, who then updates the product stock.

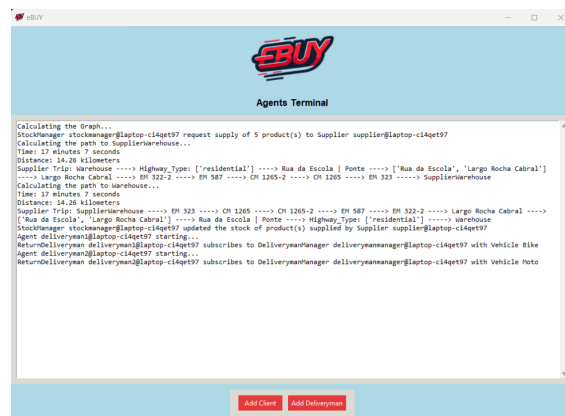


Figure 19: Supply

8.4 Negotiation

As previously mentioned, there are two distinct types of negotiations in **eBUY**: The first is between the Client and the StockManager when there is insufficient stock available for the purchase requested by the Client; the second occurs between the Supplier and StockManager when the SupplierWarehouse lacks the necessary stock.

8.4.1 Negotiation between Client and StockManager

In the image on the left, we observe the scenario where the Client accepts the StockManager's proposal, and the purchase process proceeds to the DeliverymanManager and beyond. Conversely, in the image on the right, we see the scenario where the Client rejects the proposal, resulting in the order being disregarded by the StockManager.

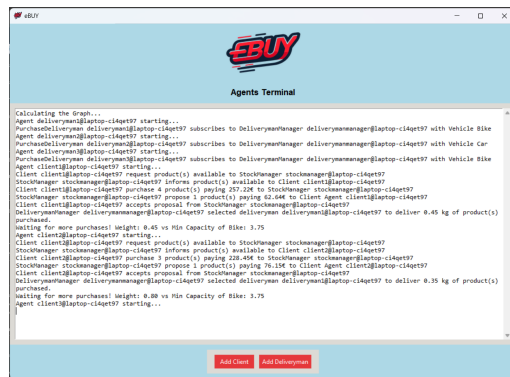


Figure 20: Client - Accept Proposal

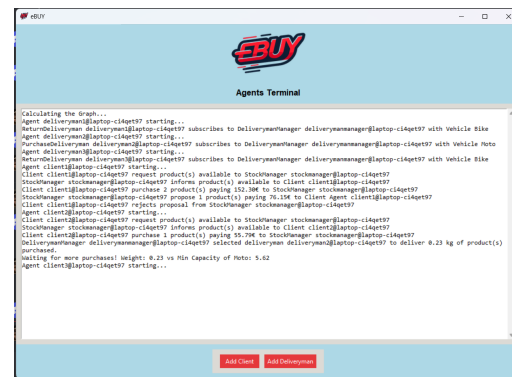


Figure 21: Client - Reject Proposal

8.4.2 Negotiation between Supplier and StockManager

In the image on the left, we see the scenario where the StockManager accepts the Supplier's proposal and the Supplier then returns to the Warehouse allowing the StockManager to update the stock. Conversely, in the image on the right, we see the scenario where the StockManager rejects the proposal and the Supplier returns to the Warehouse without any products.

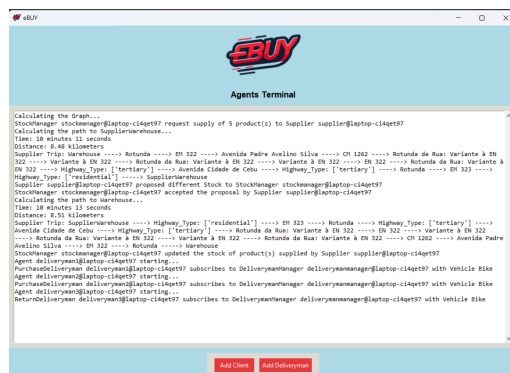


Figure 22: Supplier - Accept Proposal

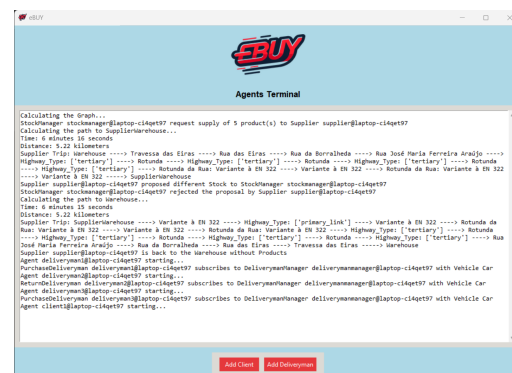


Figure 23: Supplier - Reject Proposal

8.5 Stats

The statistical overview showcased in the images distinctly highlights the operational achievements and areas of activity within the **eBUY** system across two distinct periods.

Initially, as depicted in the image on the left, there were no products returned and twenty-four products were bought — eighteen in fitness gear, four in clothing and two in supplements, with no purchases registered in other categories. The total weight of products scheduled for delivery was 2.42 kg and 232.07 kg had already been delivered.

In contrast, the period represented in the image on the right shows that five products were returned and thirty-five products were purchased. The total weight of products delivered amounted to 241.76 kg and no products were pending delivery, as indicated by the absence of records in the “to be delivered” section.

```
Stats DeliverymanManager:
- Total weight of products to be delivered: 2.42 kg
- Total weight of products delivered: 232.07 kg
- Client with most deliveries: client@laptop-ci4qet97 with 1 deliveries
- Available Purchase Deliveryman: 2

Stats StockManager:
- Category: Fitness Gear: Bought - Total quantity: 18, Total price: $448.00, Returned - Total quantity: 0, Total price: $0.00
- Category: Clothing: Bought - Total quantity: 4, Total price: $306.00, Returned - Total quantity: 0, Total price: $0.00
- Category: Supplements: Bought - Total quantity: 2, Total price: $136.36, Returned - Total quantity: 0, Total price: $0.00
- Total products bought across all categories: 24
- Total products returned across all categories: 0
```

Figure 24: Stats - Moment 1

```
Stats DeliverymanManager:
- Total weight of products delivered: 241.76 kg
- Number of products returned: 5.00 kg
- Client with most deliveries: client@laptop-ci4qet97 with 2 deliveries
- Available Purchase Deliveryman: 2

Stats StockManager:
- Category: Fitness Gear: Bought - Total quantity: 13, Total price: $309.00, Returned - Total quantity: 5, Total price: $143.00
- Category: Clothing: Bought - Total quantity: 6, Total price: $356.00, Returned - Total quantity: 0, Total price: $0.00
- Category: Supplements: Bought - Total quantity: 3, Total price: $25.40, Returned - Total quantity: 0, Total price: $0.00
- Category: Drinks: Bought - Total quantity: 2, Total price: $62.34, Returned - Total quantity: 0, Total price: $0.00
- Category: Food: Bought - Total quantity: 5, Total price: $26.12, Returned - Total quantity: 0, Total price: $0.00
- Category: Health: Bought - Total quantity: 4, Total price: $26.54, Returned - Total quantity: 0, Total price: $0.00
- Total products bought across all categories: 35
- Total products returned across all categories: 5
```

Figure 25: Stats - Moment 2

8.6 Addition of Client and Deliveryman

As illustrated in the following two images, it is feasible to add agents in real-time to the **eBUY** system. The image on the left shows the addition of a new client (client4), while the image on the right depicts the introduction of a new deliveryman (deliveryman4). This capability demonstrates the system’s flexibility and scalability, allowing for dynamic adjustments to meet evolving operational needs.

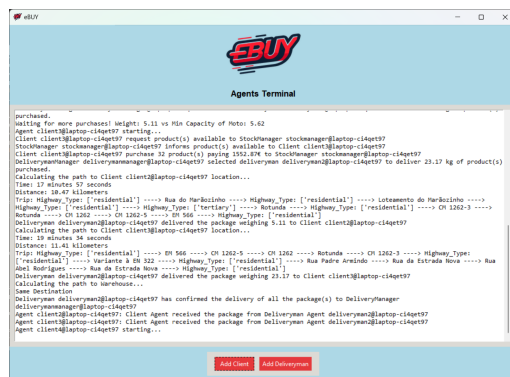


Figure 26: Add Client

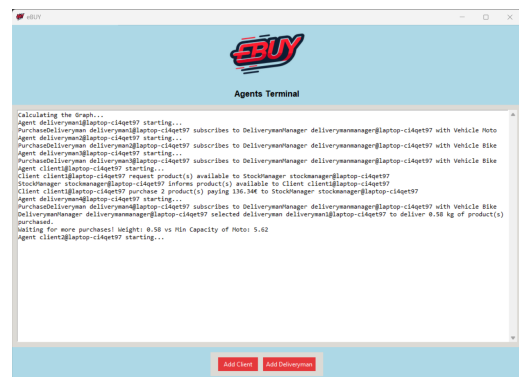


Figure 27: Add Deliveryman

9 Critical Analysis and Future Improvements

Despite the high complexity of the **eBUY**, there are always details that can be refined to make the system more realistic and functional. The team identified several potential enhancements that could be implemented to improve the project’s efficiency and user experience.

The team attempted to implement threading for the Client and Deliveryman components to adopt a distributed architecture. However, despite considerable efforts, these attempts were unsuccessful. Implementing threading for each delivery and client would have allowed the system to process multiple requests simultaneously, significantly enhancing throughput and responsiveness.

Another proposed improvement focuses on enhancing the Graphical User Interface. This would involve integrating more comprehensive and interactive elements to provide users with a more intuitive and informative experience. Such enhancements could include real-time tracking of deliveries, a more detailed visualization of inventory levels and improved navigation and accessibility features.

Another enhancement we would like to implement involves the route calculation for deliveries. Instead of merely determining the best route from point X to point Y, we aim to expand the algorithm to define the optimal routes for all pending deliveries. However, this modification would significantly increase the complexity and the computation time required to produce results, presenting considerable challenges.

These improvements are aimed at not only augmenting the system's performance and scalability but also at enhancing the overall user engagement and satisfaction.

10 Conclusion

Throughout this project, we endeavored to replicate the delivery and return systems, as well as the stock management processes, of **Prozis** [1], a Portuguese brand specializing in sports supplements.

The **eBUY** platform began as a modest concept and was progressively refined to achieve a realistic implementation. This journey not only enhanced our practical skills but also deepened our understanding of complex systems in a real-world context.

The final system comprises five agents, twelve behaviors, and eleven classes, facilitating an intricate exchange of numerous performatives among them. Each component — Agents, Behaviors, and Classes — was meticulously designed to fulfill specific roles within the system, ensuring robust and efficient operations.

Significant attention was given to the integration of the OSMnx library, which provided the tools necessary for realistic and functional route mapping. This was crucial for optimizing delivery routes and managing logistical challenges effectively.

Regarding algorithms, the project allowed us to explore and implement various search algorithms like A* and Greedy algorithms, enhancing our understanding of their mechanics and suitability for different tasks within our system. The A* algorithm proved particularly valuable in navigating complex route planning challenges, demonstrating its superiority in achieving optimal solutions over simpler, faster algorithms like Greedy when accuracy and efficiency are paramount.

In summary, this project not only facilitated the creation of a multi-agent system capable of handling all operations related to purchases, returns, and stock management using SPADE but also deepened our theoretical knowledge of search algorithms. More importantly, it highlighted the significance of applying these algorithms in real-world scenarios, thereby bridging the gap between theoretical constructs and their practical applications in industry-specific technologies.

References

- [1] <https://www.prozis.com/pt/pt>, [Accessed 10-05-2024]
- [2] Boeing, G.: Osmnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems* **65**, 126–139 (2017). <https://doi.org/https://doi.org/10.1016/j.compenvurbsys.2017.05.004>, <https://www.sciencedirect.com/science/article/pii/S0198971516303970>
- [3] Hagberg, A.A., Schult, D.A., Swart, P.J.: Exploring network structure, dynamics, and function using networkx. In: Varoquaux, G., Vaught, T., Millman, J. (eds.) *Proceedings of the 7th Python in Science Conference*. pp. 11 – 15. Pasadena, CA USA (2008)